

Politecnico di Torino
International University

Politecnico di Torino

Master of Science in Computer Engineering
Department of Control and Computer Engineering (DAUIN)

Large Language Models for Software Engineering

Laboratory 03: BERT and GPT-2

Complete notebook-aligned solution

Course Instructor
Prof. Flavio Giobergia

Prepared by
Nicola Beccaceci

Laboratory Assistant
Claudio Savelli

Academic Year
2026/2027

Contents

1 BERT: Bidirectional Encoder Representations from Transformers **1**
- 1.1 Encoder structure and representation objective . . . 1
- 1.2 Tokenization and input structure . . . 3
- 1.3 Learned embeddings and parameter scale . . . 6
- 1.4 Masked language modeling and next-sentence prediction . . . 8
- 1.5 Bidirectional attention . . . 11
- 1.6 Sentiment fine-tuning on IMDB . . . 14

2 GPT-2: Causal Language Modeling and Text Generation **20**
- 2.1 Model structure, causal attention, and positions . . . 20
- 2.2 Autoregressive generation policies . . . 25

CHAPTER 1

BERT: Bidirectional Encoder Representations from Transformers

This exercise examines BERT as a bidirectional encoder and applies its pretrained representations to sentiment classification. The objective is to trace the route from text to WordPiece IDs, through learned embeddings and encoder layers, into masked-token and sentence-pair predictions, attention maps, and a fine-tuned classifier. The notebook uses the pretrained `bert-base-uncased` checkpoint, then trains a two-class model on a seeded subset of IMDB. Success is demonstrated by correct tensor dimensions and task-head behavior, interpretable attention and position visualizations, and held-out evaluation metrics after fine-tuning. The masked-language and next-sentence objectives are introduced first because they explain the heads inspected later; this laboratory runs inference with those pretrained heads rather than repeating BERT's original pretraining.

1.1 Encoder structure and representation objective

BERT's encoder lets each input position attend to all positions in the same sequence. This bidirectionality supports contextual representations in which the interpretation of a token can depend on both its left and right context. During original pretraining, two objectives shaped these representations: masked language modeling (MLM) predicts selected tokens from their surrounding context, and next sentence prediction (NSP) classifies whether one segment follows another. BERT selected 15% of token positions for MLM prediction, with the corruption procedure retaining some selected words unchanged or replacing them with random tokens as well as '[MASK]'; the loss is applied at the selected positions. NSP contrasts an observed continuation with a sentence sampled as a negative pair. The lecture's architectural contrast is that BERT can use the full sentence in both directions, whereas a decoder-only model is restricted to its prefix (00:08:39–00:12:41). The notebook demonstrates both objectives with a pretrained checkpoint. It then uses the same encoder for supervised sentiment classification, where a newly initialized classification head is trained on labeled reviews.

This first inspection loads the pretrained base encoder and prints its module tree. It establishes the shapes and component names used throughout the exercise: token, position, and segment embeddings feed twelve encoder layers, each with multi-head attention and a feed-forward sublayer. The pooler is part of this checkpoint interface; it does not change the bidirectional attention mask.

```
from transformers import BertModel, BertTokenizer
import torch
import matplotlib.pyplot as plt

model_name = "bert-base-uncased"

# Load pre-trained BERT model
model = BertModel.from_pretrained(model_name)
```

```
print(model)
```

```
[transformers] BertModel LOAD REPORT from: bert-base-uncased
Key | Status | |
------------------------------------------+------------+--+-
cls.seq_relationship.bias | UNEXPECTED | |
cls.predictions.transform.dense.bias | UNEXPECTED | |
cls.predictions.transform.LayerNorm.bias | UNEXPECTED | |
cls.predictions.transform.LayerNorm.weight | UNEXPECTED | |
cls.predictions.bias | UNEXPECTED | |
cls.seq_relationship.weight | UNEXPECTED | |
cls.predictions.transform.dense.weight | UNEXPECTED | |

Notes:
- UNEXPECTED: can be ignored when loading from different task/architecture; not ok if
    you expect identical arch.
BertModel(
  (embeddings): BertEmbeddings(
    (word_embeddings): Embedding(30522, 768, padding_idx=0)
    (position_embeddings): Embedding(512, 768)
    (token_type_embeddings): Embedding(2, 768)
    (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True, bias=True)
    (dropout): Dropout(p=0.1, inplace=False)
  )
  (encoder): BertEncoder(
    (layer): ModuleList(
      (0-11): 12 x BertLayer(
        (attention): BertAttention(
          (self): BertSelfAttention(
            (query): Linear(in_features=768, out_features=768, bias=True)
            (key): Linear(in_features=768, out_features=768, bias=True)
            (value): Linear(in_features=768, out_features=768, bias=True)
            (dropout): Dropout(p=0.1, inplace=False)
          )
          (output): BertSelfOutput(
            (dense): Linear(in_features=768, out_features=768, bias=True)
            (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True, bias=True)
            (dropout): Dropout(p=0.1, inplace=False)
          )
        )
        (intermediate): BertIntermediate(
          (dense): Linear(in_features=768, out_features=3072, bias=True)
          (intermediate_act_fn): GELUActivation()
        )
        (output): BertOutput(
          (dense): Linear(in_features=3072, out_features=768, bias=True)
          (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True, bias=True)
          (dropout): Dropout(p=0.1, inplace=False)
        )
      )
    )
  )
  (pooler): BertPooler(
    (dense): Linear(in_features=768, out_features=768, bias=True)
    (activation): Tanh()
  )
```

```
)
```

The next cell isolates one encoder block so that the model summary can be read at the level of query, key, value projections, residual paths, normalization, and the 768-to-3072-to-768 feed-forward transformation. This inspection reports architecture rather than evaluating a sentence.

```
# Print the first layer of the BERT model
print(model.encoder.layer[0])
```

```
BertLayer(
  (attention): BertAttention(
    (self): BertSelfAttention(
      (query): Linear(in_features=768, out_features=768, bias=True)
      (key): Linear(in_features=768, out_features=768, bias=True)
      (value): Linear(in_features=768, out_features=768, bias=True)
      (dropout): Dropout(p=0.1, inplace=False)
    )
    (output): BertSelfOutput(
      (dense): Linear(in_features=768, out_features=768, bias=True)
      (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True, bias=True)
      (dropout): Dropout(p=0.1, inplace=False)
    )
  )
  (intermediate): BertIntermediate(
    (dense): Linear(in_features=768, out_features=3072, bias=True)
    (intermediate_act_fn): GELUActivation()
  )
  (output): BertOutput(
    (dense): Linear(in_features=3072, out_features=768, bias=True)
    (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True, bias=True)
    (dropout): Dropout(p=0.1, inplace=False)
  )
)
```

1.2 Tokenization and input structure

A Transformer receives integer IDs rather than raw words. BERT's WordPiece tokenizer decomposes infrequent words into vocabulary pieces; continuation pieces use the '##' prefix, so '##end' is distinct from 'end' at a word boundary. For a single sentence, the tokenizer wraps the sequence with '[CLS]' and '[SEP]'; for paired text, '[SEP]' marks the boundary and token-type IDs distinguish the segments. Padding and truncation make batch dimensions regular, with the attention mask indicating which positions are actual input rather than padding.

The tokenizer maps a simple sentence to WordPiece units and then to integer vocabulary IDs. The resulting alignment shows that model inputs are token IDs, while the readable token list is a diagnostic view of the segmentation.

```
# Load pre-trained tokenizer
tokenizer = BertTokenizer.from_pretrained(model_name)

sentence = "hello, this is a sentence!"

# Tokenize the sentence
tokens = tokenizer.tokenize(sentence)
print("Tokens:", tokens)

# Convert tokens to IDs
```

```
token_ids = tokenizer.convert_tokens_to_ids(tokens)
print("Token IDs:", token_ids)
```

```
Tokens: ['hello', ',', 'this', 'is', 'a', 'sentence', '!']
Token IDs: [7592, 1010, 2023, 2003, 1037, 6251, 999]
```

A less common word makes the subword rule visible: the word "superintend" is decomposed into the initial piece 'super' and continuation pieces '##int' and '##end'. Its integer sequence preserves the word's order while allowing rare forms to be represented using the fixed vocabulary.

```
sentence = "it was her responsibility to superintend the entire construction process"

# Tokenize the sentence
tokens = tokenizer.tokenize(sentence)
print("Tokens:", tokens)

# Convert tokens to IDs
token_ids = tokenizer.convert_tokens_to_ids(tokens)
print("Token IDs:", token_ids)
```

```
Tokens: ['it', 'was', 'her', 'responsibility', 'to', 'super', '##int', '##end', 'the',
     'entire', 'construction', 'process']
Token IDs: [2009, 2001, 2014, 5368, 2000, 3565, 18447, 10497, 1996, 2972, 2810, 2832]
```

This lookup compares a continuation token with the same spelling when it begins a word. Their distinct IDs confirm that the '##' marker is part of the token identity, not merely display punctuation.

```
tokenizer.convert_tokens_to_ids(["##end", "end"])
```

```
[10497, 2203]
```

Single-sentence encoding adds '[CLS]' and '[SEP]', converts tokens to IDs, and pads to a fixed length of ten. The tensor dimension is therefore one example by ten positions; the padding mask is available in the returned dictionary even though this cell prints IDs only.

```
sentence1 = "This is a short sentence."

# Tokenize the sentence with padding=True and max_length=10
tokens_single = tokenizer(sentence1, return_tensors="pt", padding="max_length", max_length=10)

# Display tokenized output and input IDs
print("Single Sentence Tokenization:")
print("Tokens:", tokenizer.convert_ids_to_tokens(tokens_single['input_ids'][0]))
print("Input IDs:", tokens_single['input_ids'][0])
```

```
Single Sentence Tokenization:
Tokens: ['[CLS]', 'this', 'is', 'a', 'short', 'sentence', '.', '[SEP]', '[PAD]', '[PAD
    ]']
Input IDs: tensor([ 101, 2023, 2003, 1037, 2460, 6251, 1012, 102, 0, 0])
```

Pair encoding extends the same procedure to two segments. The tokenizer inserts a separator between the sentences and pads the result to twenty positions; truncation is enabled so that inputs exceeding this bound are shortened consistently.

```
sentence2 = "But now it is a little longer."

# Tokenize the two sentences with padding and max_length of 20
# (You can pass a pair of sentences to the tokenizer as the first and
```

```
# second argument, they will be treateed as a pair of sentences separated by [SEP])
tokens_pair = tokenizer(sentence1, sentence2, return_tensors="pt", padding="max_length", max_length=20,
    truncation=True)

# Display tokenized output and input IDs
print("\nTwo Sentences Tokenization:")
print("Tokens:", tokenizer.convert_ids_to_tokens(tokens_pair['input_ids'][0]))
print("Input IDs:", tokens_pair['input_ids'][0])
```

```
Two Sentences Tokenization:
Tokens: ['[CLS]', 'this', 'is', 'a', 'short', 'sentence', '.', '[SEP]', 'but', 'now',
    'it', 'is', 'a', 'little', 'longer', '.', '[SEP]', '[PAD]', '[PAD]', '[PAD]']
Input IDs: tensor([ 101, 2023, 2003, 1037, 2460, 6251, 1012, 102, 2021, 2085, 2009,
    2003,
      1037, 2210, 2936, 1012, 102, 0, 0, 0])
```

The segment-ID tensor marks the first sentence and its boundary with zero, then the second sentence and its boundary with one; padded positions use zero. BERT adds the corresponding learned token-type vector to each token representation.

```
tokens_pair["token_type_ids"]
```

```
tensor([[0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0]])
```

The final tokenizer probe exposes the mask token's vocabulary ID in context. '[MASK]' is a dedicated token (ID 103), allowing the masked-language head to associate a specific position with a vocabulary prediction.

```
# show a mask token
mask_token = tokenizer.mask_token
mask_token_id = tokenizer.convert_tokens_to_ids(mask_token)

# Encode a text
text = "I do not know how the sentence will [MASK]."

# Tokenize the text
tokens = tokenizer.tokenize(text)

# Convert tokens to IDs
token_ids = tokenizer.convert_tokens_to_ids(tokens)

# Print each token and its ID
for token, token_id in zip(tokens, token_ids):
    print(f"Token: {token}, ID: {token_id}")
```

```
Token: i, ID: 1045
Token: do, ID: 2079
Token: not, ID: 2025
Token: know, ID: 2113
Token: how, ID: 2129
Token: the, ID: 1996
Token: sentence, ID: 6251
Token: will, ID: 2097
Token: [MASK], ID: 103
Token: ., ID: 1012
```

1.3 Learned embeddings and parameter scale

Because self-attention has no recurrent state that encodes order, BERT adds a learned absolute position vector and a learned segment vector to each token embedding. The three vectors have the same width and are summed position by position before entering the encoder. Inspecting cosine similarity shows the geometry learned for positions, while the parameter count places these tables in the context of the full 110-million-parameter BERT-base checkpoint. The model was pretrained mostly with sequence length 128 and later exposed to length 512, a schedule that helps explain why the first part of the position-similarity matrix can look more structured.

These imports supply the numerical and plotting operations for inspecting the learned position table. The cell intentionally produces no output; it prepares the functions used by the following extraction and visualization.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.metrics.pairwise import cosine_similarity
from transformers import BertModel
```

> This cell intentionally has no displayed output. Its state change is described above and is used by subsequent cells.

This cell extracts the (512 imes768) learned position matrix, computes cosine similarity for every pair of positions, and plots both the full matrix and the row for position 100. The diagonal is one by definition; structure away from the diagonal indicates which learned position vectors point in similar directions.

```
# Load pre-trained BERT model
model_name = 'bert-base-uncased'
model = BertModel.from_pretrained(model_name)

# Position embeddings are stored in the embedding layer as 'position_embeddings.weight'
positional_encodings = model.embeddings.position_embeddings.weight.detach().cpu().numpy()

# Compute cosine similarity between positional encodings
cosine_sim = cosine_similarity(positional_encodings)

fig, ax = plt.subplot_mosaic("""
AAA
AAA
AAA
BBB""")

plt.tight_layout()

# Plot the cosine similarity heatmap
ax["A"].set_aspect('equal')
cbar = ax["A"].imshow(cosine_sim, cmap='viridis', vmin=-1, vmax=1)
ax["A"].set_xlabel('Vector 1')
ax["A"].set_ylabel('Vector 2')
ax["A"].axhline(100, c='r', lw=2)
# add colorbar
fig.colorbar(cbar, ax=ax["A"])


# Plot the cosine similarity for a specific row

ax["B"].plot(cosine_sim[100], c='r', lw=2)
ax["B"].set_xlabel('Vector 1')
ax["B"].set_ylabel('Cosine Similarity')
ax["B"].grid()
```

```
[transformers] BertModel LOAD REPORT from: bert-base-uncased
Key | Status | |
```

```
-----------------------------------------+------------+--+-
cls.seq_relationship.bias | UNEXPECTED | |
cls.predictions.transform.dense.bias | UNEXPECTED | |
cls.predictions.transform.LayerNorm.bias | UNEXPECTED | |
cls.predictions.transform.LayerNorm.weight | UNEXPECTED | |
cls.predictions.bias | UNEXPECTED | |
cls.seq_relationship.weight | UNEXPECTED | |
cls.predictions.transform.dense.weight | UNEXPECTED | |

Notes:
- UNEXPECTED: can be ignored when loading from different task/architecture; not ok if
    you expect identical arch.
```

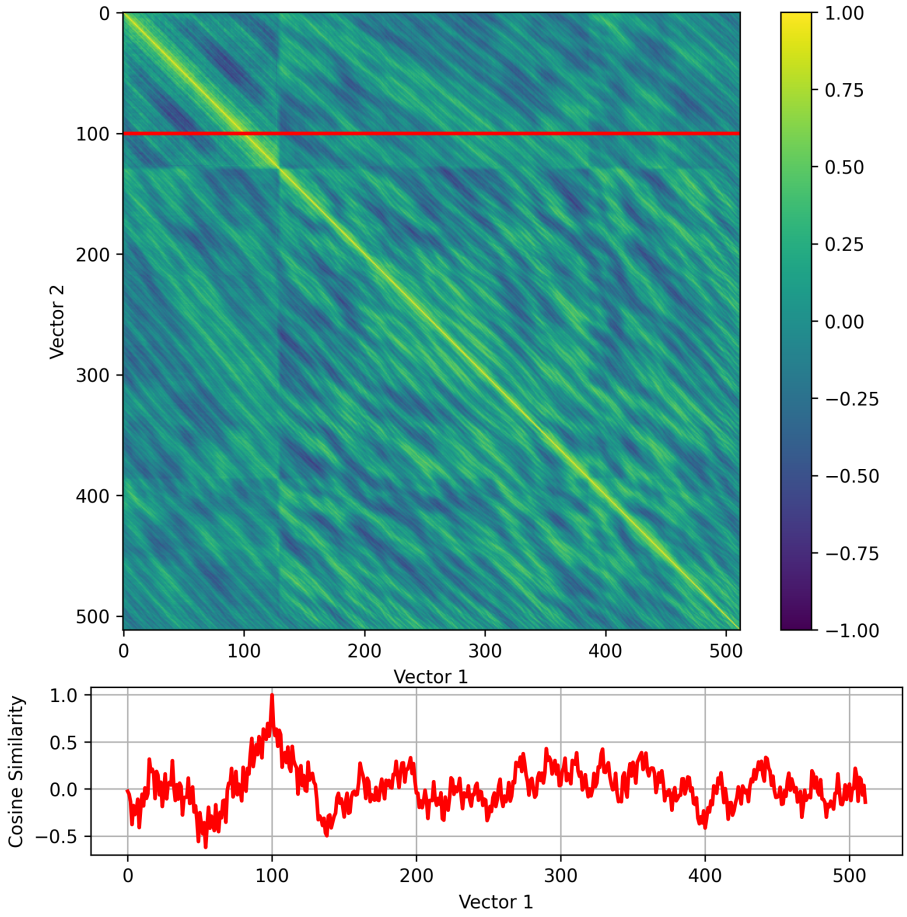


Figure 1.1: Learned BERT positional vectors: pairwise cosine similarity and the position-100 cross-section.

The diagonal confirms identical-position similarity; the off-diagonal structure is learned rather than imposed by a sinusoidal formula. The 128-position crop is discussed next because most pretraining updates used that length.

The crop focuses on the first 128 positions, which were used for most BERT pretraining steps. Comparing it with the full matrix makes the more regular local structure easier to inspect. The stored plot is the notebook's output, not a newly generated estimate.

```
fig, ax = plt.subplots()

ax.set_aspect('equal')
cbar = ax.imshow(cosine_sim[:128, :128], cmap='viridis', vmin=-1, vmax=1)
ax.set_xlabel('Vector 1')
```

```
ax.set_ylabel('Vector 2')
# add colorbar
fig.colorbar(cbar, ax=ax)
```

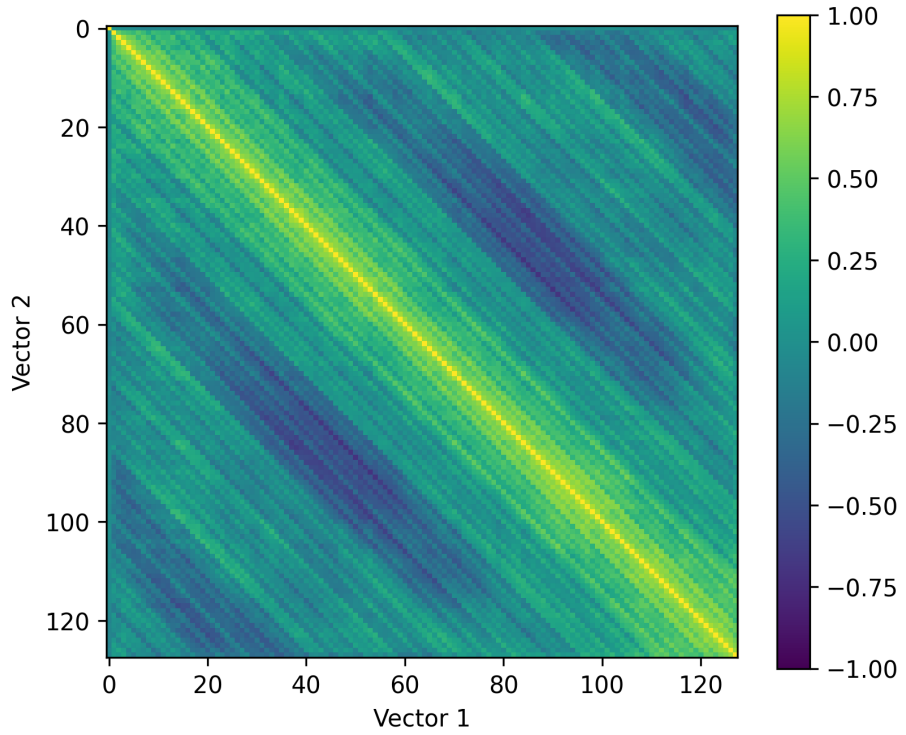


Figure 1.2: Cosine-similarity submatrix for positions 0–127.

The early-position pattern is more structured than the full 512-position matrix, consistent with the notebook's note that most pretraining steps used length 128. Similarity alone does not reveal the training objective or guarantee that positions have interchangeable roles.

The token-type table contains two learned vectors, each 768 dimensions wide. This matches the binary segment IDs produced for sentence-pair tasks.

```
model.embeddings.token_type_embeddings.weight.shape
```

```
torch.Size([2, 768])
```

Counting model parameters verifies the scale of the loaded BERT-base encoder, including embeddings, attention and feed-forward layers, normalization, and the pooler. The total is a checkpoint-level count; a task-specific head may add parameters in later sections.

```
sum([ p.numel() for p in model.parameters () ])
```

```
109482240
```

1.4 Masked language modeling and next-sentence prediction

For MLM, the encoder produces a hidden vector at every input position; a prediction head maps each vector to 30,522 vocabulary logits. At training time, loss is evaluated at selected masked positions. The inference example below places one '[MASK]' token in a sentence, extracts

its score vector, and displays the five highest-scoring words. NSP instead reads the pooled representation of a pair and predicts whether the second segment is the observed continuation. Together the examples show the purpose and output form of BERT's original pretraining heads, while their single examples are demonstrations rather than estimates of general accuracy.

This import prepares the two pretrained task heads used in the next section. It changes no model state and intentionally has no displayed output.

```
from transformers import BertForMaskedLM, BertForNextSentencePrediction
import torch
```

This cell intentionally has no displayed output. Its state change is described above and is used by subsequent cells.

The masked-language model class adds a vocabulary projection to the encoder. Printing 'model_mlm.cls' shows the transformation and normalization followed by a decoder with 30,522 outputs, one score per WordPiece vocabulary item.

```
# Load pre-trained BERT model and tokenizer
model_name = 'bert-base-uncased'

# Load pre-trained BERT model for masked language modeling
model_mlm = BertForMaskedLM.from_pretrained(model_name)

print(model_mlm.cls)
```

```
[transformers] BertForMaskedLM LOAD REPORT from: bert-base-uncased
Key | Status | |
----------------------------+------------+--+-
bert.pooler.dense.bias | UNEXPECTED | |
bert.pooler.dense.weight | UNEXPECTED | |
cls.seq_relationship.bias | UNEXPECTED | |
cls.seq_relationship.weight | UNEXPECTED | |

Notes:
- UNEXPECTED: can be ignored when loading from different task/architecture; not ok if
    you expect identical arch.
BertOnlyMLMHead(
  (predictions): BertLMPredictionHead(
    (transform): BertPredictionHeadTransform(
      (dense): Linear(in_features=768, out_features=768, bias=True)
      (transform_act_fn): GELUActivation()
      (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True, bias=True)
    )
    (decoder): Linear(in_features=768, out_features=30522, bias=True)
  )
)
```

The sentence contains a single mask. Tokenization returns one batch of nine positions, including '[CLS]', '[SEP]', and punctuation; inference under 'no_grad' returns logits at every position. Only the row at the masked position will be used for prediction.

```
sentence = "I want to eat a [MASK]."

# Tokenize the sentence
inputs = tokenizer(sentence, return_tensors="pt")
print(inputs["input_ids"])

with torch.no_grad():

    # Get the model outputs
```

```
    outputs = model_mlm(**inputs)
    predictions = outputs.logits

print(predictions.shape)
```

```
tensor([[ 101, 1045, 2215, 2000, 4521, 1037, 103, 1012, 102]])
torch.Size([1, 9, 30522])
```

The output shape (1, 9, 30,522) has one vocabulary score vector for each of the nine sequence positions. The task loss would select masked positions during pretraining; this inference example instead locates the one explicit ‘[MASK]‘ token.

The mask selects the vocabulary logits for the masked position, and sorting returns the five highest-scoring token IDs. These are model scores for one context, not calibrated probabilities; their ordering is the relevant result for this top-five inspection.

```
top_k = 5
mask = (inputs['input_ids'] == tokenizer.mask_token_id)
top_k_tokens = torch.argsort(predictions[mask], descending=True)[0, :top_k] # 0 => only take first mask

for token_id, token, logit in zip(top_k_tokens, tokenizer.convert_ids_to_tokens(top_k_tokens), predictions[
    mask][0, top_k_tokens]):
    print(f"{token} ({token_id}) - logit: {logit:.4f}")
```

```
little (2210) - logit: 10.2418
bite (6805) - logit: 9.8704
burger (15890) - logit: 9.0154
lot (2843) - logit: 8.9455
sandwich (11642) - logit: 8.6518
```

Next-sentence prediction takes two segments and returns logits for the binary relation label. Here the second sentence continues the topic of the first, so the high probability for the ‘IsNext‘ class illustrates the checkpoint’s learned NSP head. This is an inference demonstration, not a new pretraining run.

```
from transformers import BertForNextSentencePrediction
model_nsp = BertForNextSentencePrediction.from_pretrained(model_name)

sentence_a = "Arthur Conan Doyle is my favorite author"
sentence_b = "I read all of his books"

# Tokenize the two sentences
inputs_nsp = tokenizer(sentence_a, sentence_b, return_tensors="pt")

print("Tokens:", tokenizer.convert_ids_to_tokens(inputs_nsp['input_ids'][0]))

with torch.no_grad():

    # Get the model outputs
    outputs_nsp = model_nsp(**inputs_nsp)
    logits = outputs_nsp.logits
    probs = torch.softmax(logits, dim=-1)

print(f"True: {probs[0,0].item():.6f}, False: {probs[0,1].item():.6f}")
```

```
[transformers] BertForNextSentencePrediction LOAD REPORT from: bert-base-uncased
Key | Status | |
------------------------------------------+------------+--+-
cls.predictions.transform.dense.bias | UNEXPECTED | |
cls.predictions.transform.LayerNorm.bias | UNEXPECTED | |
cls.predictions.transform.LayerNorm.weight | UNEXPECTED | |
cls.predictions.bias | UNEXPECTED | |
```

```
cls.predictions.transform.dense.weight | UNEXPECTED | |

Notes:
- UNEXPECTED: can be ignored when loading from different task/architecture; not ok if
    you expect identical arch.
Tokens: ['[CLS]', 'arthur', 'conan', 'doyle', 'is', 'my', 'favorite', 'author', '[SEP
    ]', 'i', 'read', 'all', 'of', 'his', 'books', '[SEP]']
True: 0.999989, False: 0.000011
```

The two probabilities sum to one because the cell applies softmax across the two relation classes. Their contrast demonstrates the head's response to these examples, although a pretrained probability is not proof of a sentence relation.

The second pair changes the context to an unrelated weather sentence. The tokenizer preserves the same segment structure, while the NSP head assigns most probability to 'NotNext'. These probabilities describe this pretrained model's judgment and should not be read as a general measure of semantic coherence.

```
# Tokenize the two sentences
sentence_c = "However, it's raining today"
inputs_nsp_false = tokenizer(sentence_a, sentence_c, return_tensors="pt")

print("Tokens:", tokenizer.convert_ids_to_tokens(inputs_nsp_false['input_ids'][0]))

with torch.no_grad():

    # Get the model outputs
    outputs_nsp = model_nsp(**inputs_nsp_false)
    logits = outputs_nsp.logits
    probs = torch.softmax(logits, dim=-1)

print(f"True: {probs[0,0].item():.6f}, False: {probs[0,1].item():.6f}")
```

```
Tokens: ['[CLS]', 'arthur', 'conan', 'doyle', 'is', 'my', 'favorite', 'author', '[SEP
    ]', 'however', ',', 'it', "'", 's', 'raining', 'today', '[SEP]']
True: 0.004302, False: 0.995698
```

The two probabilities sum to one because the cell applies softmax across the two relation classes. Their contrast demonstrates the head's response to these examples, although a pretrained probability is not proof of a sentence relation.

1.5 Bidirectional attention

With attention output enabled, each BERT layer returns one matrix per head. For a sequence of length T, each matrix has shape $T \times T$; row i describes how the query at position i distributes weight across key positions. The notebook first reports these dimensions, then plots one head and all layer-head combinations for the sentence "The dog ate the food because it was hungry." The displays can suggest which tokens interact, but attention weights alone do not establish why a prediction was made.

The imports provide the attention-enabled model, tokenizer, tensor utilities, and plotting functions. This setup cell has no output because it only establishes names for the next forward pass.

```
from transformers import BertModel, BertTokenizer
import torch
import matplotlib.pyplot as plt
import numpy as np
```

This cell intentionally has no displayed output. Its state change is described above and is used by subsequent cells.

The encoder is loaded with attention reporting enabled, then processes a short sentence in one forward pass. The returned ‘attentions‘ tuple stores one tensor per layer; no prediction head is involved in this representation inspection.

```
# Load pre-trained BERT model (with ‘output_attentions=True‘) and tokenizer
model = BertModel.from_pretrained(model_name, output_attentions=True)
tokenizer = BertTokenizer.from_pretrained(model_name)

sentence = "The dog ate the food because it was hungry"

# Tokenize the sentence
inputs = tokenizer(sentence, return_tensors="pt")

# Get the model outputs
outputs = model(**inputs)

attention = outputs.attentions  # This returns a list of attention matrices
```

```
[transformers] BertModel LOAD REPORT from: bert-base-uncased
Key | Status | |
------------------------------------------+------------+--+-
cls.seq_relationship.bias | UNEXPECTED | |
cls.predictions.transform.dense.bias | UNEXPECTED | |
cls.predictions.transform.LayerNorm.bias | UNEXPECTED | |
cls.predictions.transform.LayerNorm.weight | UNEXPECTED | |
cls.predictions.bias | UNEXPECTED | |
cls.seq_relationship.weight | UNEXPECTED | |
cls.predictions.transform.dense.weight | UNEXPECTED | |

Notes:
- UNEXPECTED: can be ignored when loading from different task/architecture; not ok if
    you expect identical arch.
```

The shape report confirms twelve layers. Each layer’s tensor has dimensions (batch, heads, query positions, key positions), here (1, 12, 11, 11), so each head assigns a distribution over eleven input positions for every query token.

```
print("len(attention):", len(attention))
print("attention shape", attention[0].shape)
```

```
len(attention): 12
attention shape torch.Size([1, 12, 11, 11])
```

This plot selects layer 2, head 2 (zero-based indices 1 and 1) and labels both axes with the input tokens. A row shows where one query position places attention; the heatmap is descriptive and does not by itself establish a causal or linguistic explanation.

```
fig, ax = plt.subplots()

ax.imshow(attention[1][0, 1].detach().cpu().numpy(), cmap='viridis', aspect='equal')
ax.set_xticks(np.arange(len(inputs['input_ids'][0])), tokenizer.convert_ids_to_tokens(inputs['input_ids'
    ][0]), rotation=90)
ax.set_yticks(np.arange(len(inputs['input_ids'][0])), tokenizer.convert_ids_to_tokens(inputs['input_ids'
    ][0]));

ax.set_xlabel("Attended token")
ax.set_ylabel("Attending token")
```

```
Text(0, 0.5, 'Attending token')
```

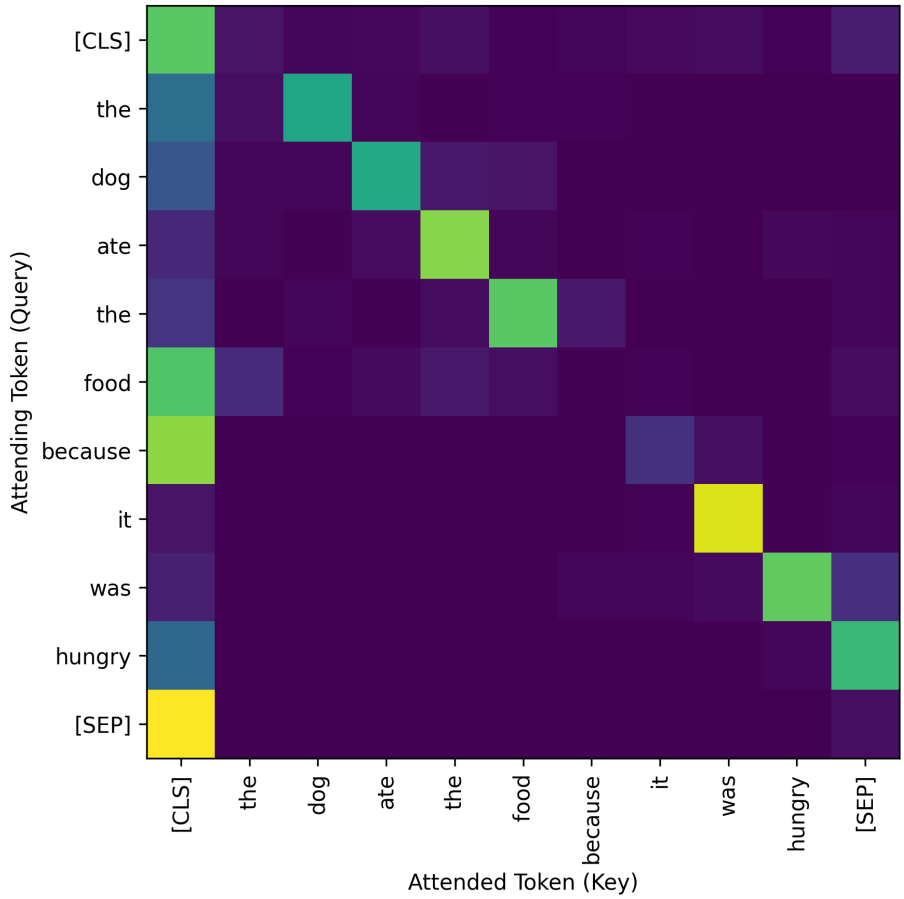


Figure 1.3: One BERT attention head for the sentence shown in the preceding code.

The rows and columns show token-to-token weights for one head and one layer. The map is a model diagnostic; interpretation should be checked against the sentence and other heads.

The grid renders all 144 layer-head matrices for the same sentence. It lets the reader compare heads at a glance; the exercise's interpretation should remain tied to visible patterns and the token labels rather than assuming every head implements a clean linguistic role.

```
fig, ax = plt.subplots(12, 12, figsize=(10, 10))

for i in range(12):
    for j in range(12):
        ax[i, j].imshow(attention[i][0, j].detach().cpu().numpy(), cmap='viridis', aspect='auto')
        ax[i, j].axis('off')
```

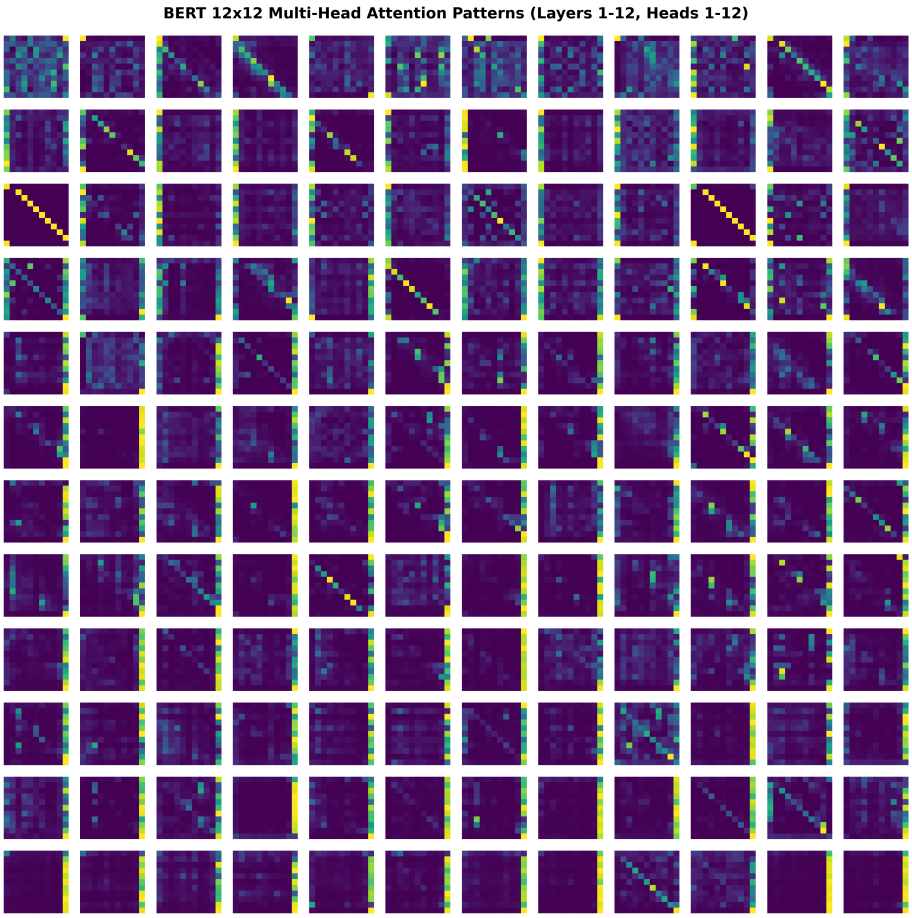


Figure 1.4: Attention maps for all 12 layers and 12 heads.

The grid confirms that heads and layers need not share one attention pattern. It is a qualitative view of the encoder's intermediate computation, not a direct explanation of its final classification decisions.

1.6 Sentiment fine-tuning on IMDB

The final exercise adapts BERT to binary review classification. 'BertForSequenceClassification' adds a two-output linear head over the pooled sequence representation; its parameters are absent from the base checkpoint and begin with a task-specific initialization. The notebook samples 2,000 training reviews and 1,000 test reviews after seeded shuffling, tokenizes each review to at most 512 positions, and uses 'Trainer' to update the model. The split and training configuration make this a bounded demonstration: its evaluation result describes the selected test subset and should not be generalized to the full dataset without a broader run.

This setup imports the sequence-classification model class and PyTorch. It has no output: its role is to prepare the model load and device selection in the next cell.

```
from transformers import BertTokenizer, BertForSequenceClassification
import torch
```

> This cell intentionally has no displayed output. Its state change is described above and is used by subsequent cells.

The base checkpoint is wrapped with a two-class classification head, and the model is moved to CUDA, Apple Metal, or CPU according to availability. The checkpoint report distinguishes

pretrained encoder weights from the newly initialized classifier parameters; those missing task-specific weights must be learned on the sentiment data.

```
# Load the pre-trained BERT model (with ‘num_labels=2‘) and tokenizer
model = BertForSequenceClassification.from_pretrained(model_name, num_labels=2)
tokenizer = BertTokenizer.from_pretrained(model_name)

# set the device to GPU if available and move the model to the device
device = torch.device("cuda" if torch.cuda.is_available() else "mps" if torch.backends.mps.is_available()
    else "cpu")
model.to(device)

print(model.classifier)
```

```
[transformers] BertForSequenceClassification LOAD REPORT from: bert-base-uncased
Key | Status |
-----------------------------------------+------------+-
cls.seq_relationship.bias | UNEXPECTED |
cls.predictions.transform.dense.bias | UNEXPECTED |
cls.predictions.transform.LayerNorm.bias | UNEXPECTED |
cls.predictions.transform.LayerNorm.weight | UNEXPECTED |
cls.predictions.bias | UNEXPECTED |
cls.seq_relationship.weight | UNEXPECTED |
cls.predictions.transform.dense.weight | UNEXPECTED |
classifier.weight | MISSING |
classifier.bias | MISSING |

Notes:
- UNEXPECTED: can be ignored when loading from different task/architecture; not ok if
    you expect identical arch.
- MISSING: those params were newly initialized because missing from the checkpoint.
    Consider training on your downstream task.
Linear(in_features=768, out_features=2, bias=True)
```

The missing classifier weights are expected because BERT-base pretraining does not include this two-label head. The printed linear layer maps each 768-dimensional ‘[CLS]‘ representation to two class scores, so fine-tuning must learn this projection along with the encoder unless layers are frozen.

The metric function converts the model’s class scores to predicted labels with ‘argmax‘ and compares them with the reference labels. It returns accuracy in the dictionary format expected by ‘Trainer‘; the function definition itself prints nothing.

```
from sklearn.metrics import accuracy_score

# Function to compute accuracy
def compute_metrics(pred):
    labels = pred.label_ids
    preds = pred.predictions.argmax(-1)
    accuracy = accuracy_score(labels, preds)
    return {"accuracy": accuracy}
```

This cell intentionally has no displayed output. Its state change is described above and is used by subsequent cells.

The IMDB dataset is loaded, then seeded shuffling selects 2,000 training and 1,000 test reviews. The fixed seed makes the subset selection reproducible relative to the dataset version, while the smaller sample bounds the demonstration’s training cost.

```
from datasets import load_dataset

# Load a sentiment analysis dataset
```

```
dataset = load_dataset('imdb')

train_dataset = dataset['train'].shuffle(seed=42).select(range(2000))
test_dataset = dataset['test'].shuffle(seed=42).select(range(1000))
```

This cell intentionally has no displayed output. Its state change is described above and is used by subsequent cells.

This cell prints one selected training record as formatted JSON. It exposes the schema (‘text‘ and ‘label‘) and a concrete review so that the mapping from dataset example to model input is explicit; the record is illustrative and is not a summary of dataset quality.

```
import json # using json to get a nicer indentation

print(json.dumps(train_dataset[0], indent=2))
```

```
{
  "text": "There is no relation at all between Fortier and Profiler but the fact that
      both are police series about violent crimes. Profiler looks crispy, Fortier
      looks classic. Profiler plots are quite simple. Fortier's plot are far more
      complicated... Fortier looks more like Prime Suspect, if we have to spot
      similarities... The main character is weak and weirdo, but have \"clairvoyance\".
       People like to compare, to judge, to evaluate. How about just enjoying? Funny
      thing too, people writing Fortier looks American but, on the other hand, arguing
       they prefer American series (!!!). Maybe it's the language, or the spirit, but
      I think this series is more English than American. By the way, the actors are
      really good and funny. The acting is not superficial at all...",
  "label": 1
}
```

The tokenizer function truncates each review to 512 tokens and returns model-ready fields. Applying it in batches to both splits creates the encoded datasets consumed by ‘Trainer‘; because the map call stores its result, this cell updates the notebook’s dataset state without printing a separate table.

```
# Tokenize the dataset
def tokenize_function(sample):
    tokenized_input = tokenizer(sample['text'], padding="max_length", truncation=True, max_length=256)
    return tokenized_input

train_dataset = train_dataset.map(tokenize_function, batched=True)
test_dataset = test_dataset.map(tokenize_function, batched=True)
```

This cell intentionally has no displayed output. Its state change is described above and is used by subsequent cells.

Training arguments set the epoch count, per-device batch sizes, evaluation and save cadence, and logging behavior. The ‘Trainer‘ binds these settings to the model, encoded datasets, tokenizer, and metric callback; constructing it prepares training but does not update weights yet.

```
from transformers import Trainer, TrainingArguments

batch_size = 32
num_train_epochs = 2

learning_rate = 2e-5
weight_decay = 0.01

# Define training arguments
training_args = TrainingArguments(
    output_dir='./results',
```

```
    eval_strategy="steps",
    eval_steps=10,
    learning_rate=learning_rate,
    per_device_train_batch_size=batch_size,
    per_device_eval_batch_size=batch_size,
    num_train_epochs=num_train_epochs,
    weight_decay=weight_decay,
    logging_steps=10,  #
)

# Initialize the Trainer object
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=train_dataset,
    eval_dataset=test_dataset,
    compute_metrics=compute_metrics
)
```

This cell intentionally has no displayed output. Its state change is described above and is used by subsequent cells.

'trainer.train()' now performs the fine-tuning run. The cell's progress and epoch logs report loss and evaluation accuracy; the saved loss/accuracy figure plots the recorded training trajectory. These results belong to the selected 2,000-review subset and the recorded hardware/software environment.

```
# Fine-tune the model
results = trainer.train()
```

```
/Users/nicolabeccaceci/miniconda3/lib/python3.13/site-packages/torch/utils/data/
    dataloader.py:759: UserWarning: 'pin_memory' argument is set as true but not
    supported on MPS now, device pinned memory won't be used.
  super().__init__(loader)
<IPython.core.display.HTML object>
/Users/nicolabeccaceci/miniconda3/lib/python3.13/site-packages/torch/utils/data/
    dataloader.py:759: UserWarning: 'pin_memory' argument is set as true but not
    supported on MPS now, device pinned memory won't be used.
  super().__init__(loader)
/Users/nicolabeccaceci/miniconda3/lib/python3.13/site-packages/torch/utils/data/
    dataloader.py:759: UserWarning: 'pin_memory' argument is set as true but not
    supported on MPS now, device pinned memory won't be used.
  super().__init__(loader)
/Users/nicolabeccaceci/miniconda3/lib/python3.13/site-packages/torch/utils/data/
    dataloader.py:759: UserWarning: 'pin_memory' argument is set as true but not
    supported on MPS now, device pinned memory won't be used.
  super().__init__(loader)
/Users/nicolabeccaceci/miniconda3/lib/python3.13/site-packages/torch/utils/data/
    dataloader.py:759: UserWarning: 'pin_memory' argument is set as true but not
    supported on MPS now, device pinned memory won't be used.
  super().__init__(loader)
/Users/nicolabeccaceci/miniconda3/lib/python3.13/site-packages/torch/utils/data/
    dataloader.py:759: UserWarning: 'pin_memory' argument is set as true but not
    supported on MPS now, device pinned memory won't be used.
  super().__init__(loader)
```

```
/Users/nicolabeccaceci/miniconda3/lib/python3.13/site-packages/torch/utils/data/
    dataloader.py:759: UserWarning: 'pin_memory' argument is set as true but not
    supported on MPS now, device pinned memory won't be used.
  super().__init__(loader)
/Users/nicolabeccaceci/miniconda3/lib/python3.13/site-packages/torch/utils/data/
    dataloader.py:759: UserWarning: 'pin_memory' argument is set as true but not
    supported on MPS now, device pinned memory won't be used.
  super().__init__(loader)
/Users/nicolabeccaceci/miniconda3/lib/python3.13/site-packages/torch/utils/data/
    dataloader.py:759: UserWarning: 'pin_memory' argument is set as true but not
    supported on MPS now, device pinned memory won't be used.
  super().__init__(loader)
/Users/nicolabeccaceci/miniconda3/lib/python3.13/site-packages/torch/utils/data/
    dataloader.py:759: UserWarning: 'pin_memory' argument is set as true but not
    supported on MPS now, device pinned memory won't be used.
  super().__init__(loader)
/Users/nicolabeccaceci/miniconda3/lib/python3.13/site-packages/torch/utils/data/
    dataloader.py:759: UserWarning: 'pin_memory' argument is set as true but not
    supported on MPS now, device pinned memory won't be used.
  super().__init__(loader)
/Users/nicolabeccaceci/miniconda3/lib/python3.13/site-packages/torch/utils/data/
    dataloader.py:759: UserWarning: 'pin_memory' argument is set as true but not
    supported on MPS now, device pinned memory won't be used.
  super().__init__(loader)
```

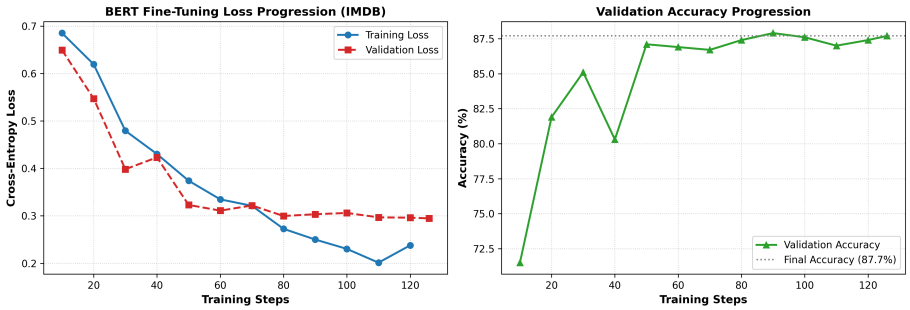


Figure 1.5: Training and evaluation metrics recorded during BERT fine-tuning.

The plot records optimization and evaluation behavior over the configured run. Training loss tracks fit to the sampled training set, whereas evaluation accuracy measures held-out predictions; neither curve by itself establishes performance on the full IMDB distribution.

Evaluation runs the trained classifier on the held-out 1,000-review subset and reports the final metrics. The result is an empirical score for this small experiment; it should be interpreted with the subset size, training setup, and label distribution in mind.

```
# Evaluate the model
results = trainer.evaluate()
print(f"Accuracy on the validation set: {results['eval_accuracy']:.4f}")
```

```
/Users/nicolabeccaceci/miniconda3/lib/python3.13/site-packages/torch/utils/data/
    dataloader.py:759: UserWarning: 'pin_memory' argument is set as true but not
    supported on MPS now, device pinned memory won't be used.
  super().__init__(loader)
<IPython.core.display.HTML object>
<IPython.core.display.HTML object>
Accuracy on the validation set: 0.8800
```

The exercise establishes how one pretrained encoder supports both self-supervised token and sentence-pair predictions and supervised classification. Its outputs also expose the distinction between pretrained encoder parameters and a newly learned task head. Attention and position figures are diagnostic views, while the final test metrics provide the direct evidence for the small sentiment experiment.

CHAPTER 2

GPT-2: Causal Language Modeling and Text Generation

This exercise studies GPT-2 as a decoder-only language model. Its objective is to inspect the learned token and position representations, observe causal self-attention, and compare decoding policies on a fixed Sherlock Holmes prompt. GPT-2 predicts the next token using only the prefix available so far; each generated token is appended to that prefix before the next prediction. The exercise succeeds when the model's dimensions and attention structure are clear and the effect of greedy, beam, random, top-k, top-p, and temperature-based decoding can be seen in their saved continuations. Generated passages are examples of model behavior, not graded language or correctness results.

2.1 Model structure, causal attention, and positions

GPT-2 shares the Transformer block pattern with BERT but applies a causal mask so that position t cannot attend to future positions. Its learned token table has 50,257 rows and its position table has 1,024 rows, each with width 768. The first saved model output belongs to the checkpoint-loading cell whose execution counter was lost in notebook metadata; its code, device output, model tree, and subsequent variable use are consistent, so the recorded state is reused without a second download. The following attention plot selects one layer and head, and the position plot examines pairwise cosine similarities among the learned vectors.

The first cell loads the GPT-2 tokenizer and language-model checkpoint, chooses the available device, and prints the model tree. Its saved output confirms the decoder's 50,257-token vocabulary, 1,024 learned positions, twelve Transformer blocks, and vocabulary projection. The missing execution counter is metadata loss: the stored output and its use by later cells provide evidence that the cell ran, so the model was not downloaded or executed again.

```
from transformers import GPT2Tokenizer, GPT2LMHeadModel
import torch

model_name = 'gpt2'

# Load GPT-2 tokenizer and model
tokenizer = GPT2Tokenizer.from_pretrained(model_name)
model = GPT2LMHeadModel.from_pretrained(model_name)
device = torch.device('cuda' if torch.cuda.is_available() else 'mps' if torch.backends.mps.is_available()
    else 'cpu')
model.to(device)

model
```

```
mps
GPT2LMHeadModel(
  (transformer): GPT2Model(
```

```
    (wte): Embedding(50257, 768)
    (wpe): Embedding(1024, 768)
    (drop): Dropout(p=0.1, inplace=False)
    (h): ModuleList(
      (0-11): 12 x GPT2Block(
        (ln_1): LayerNorm((768,), eps=1e-05, elementwise_affine=True, bias=True)
        (attn): GPT2Attention(
          (c_attn): Conv1D(nf=2304, nx=768)
          (c_proj): Conv1D(nf=768, nx=768)
          (attn_dropout): Dropout(p=0.1, inplace=False)
          (resid_dropout): Dropout(p=0.1, inplace=False)
        )
        (ln_2): LayerNorm((768,), eps=1e-05, elementwise_affine=True, bias=True)
        (mlp): GPT2MLP(
          (c_fc): Conv1D(nf=3072, nx=768)
          (c_proj): Conv1D(nf=768, nx=3072)
          (act): NewGELUActivation()
          (dropout): Dropout(p=0.1, inplace=False)
        )
      )
    )
    (ln_f): LayerNorm((768,), eps=1e-05, elementwise_affine=True, bias=True)
  )
  (lm_head): Linear(in_features=768, out_features=50257, bias=False)
)
```

The printed tree is the saved execution result for this cell. Its absent execution counter does not invalidate the captured model output: following cells use the named model and tokenizer, and the device output is ‘mps‘. The full model was reused from the notebook state rather than loaded again.

The notebook first sets the model to evaluation mode, tokenizes a Sherlock Holmes prompt, and generates up to 200 tokens with sampling at temperature 1.1. The output illustrates continuation from a long prompt; because sampling is stochastic, this exact text is one recorded draw, not a guaranteed reproduction on rerun.

```
# Set the model to evaluation mode
model.eval()

# Define the input prompt
prompt = "I had called upon my friend, Mr. Sherlock Holmes, one day in the autumn of last year and found
     him in deep conversation with a very stout, florid-faced, elderly gentleman with fiery red hair."

# Tokenize the input prompt
input_ids = tokenizer.encode(prompt, return_tensors='pt').to(device)

# Autoregressively generate tokens (‘max_length=200‘, ‘do_sample=True‘, ‘temperature=1.1‘)
generated_ids = model.generate(input_ids, max_length=200, do_sample=True, temperature=1.1, pad_token_id=
     tokenizer.eos_token_id)

# Decode and print the generated text
generated_text = tokenizer.decode(generated_ids[0], skip_special_tokens=True)

print("Generated Text:")
print(generated_text)
```

```
Generated Text:
I had called upon my friend, Mr. Sherlock Holmes, one day in the autumn of last year
    and found him in deep conversation with a very stout, florid-faced, elderly
    gentleman with fiery red hair.
```

```
"He said to him there were good people in London, that you might do well to visit
    those who are dying, and he said he would let us see them tomorrow," Holmes took
    out his clipboard, and put something on the clipboard that resembled an envelope.

"Yes, of course it would," went the address.

"The gentleman said that people of a higher caste have been dying in droves, so he
    asked me to come visit, and said it would be very good for the cause," Holmes
    continued. "The gentleman did say that if I could do this it would be his first
    day in London, a point that I thought highly of." The address continued. "We
    talked in long talk over tea, and he went out by the
```

This import cell supports the later attention and sampling code. It has no output; its saved execution count is present, and the import is also used by subsequent cells.

```
import torch
from transformers import GPT2Tokenizer, GPT2Model
import matplotlib.pyplot as plt
import seaborn as sns
```

This cell intentionally has no displayed output. Its state change is described above and is used by subsequent cells.

The base GPT-2 model returns attention tensors for an Italian-food prompt. Selecting the last layer and head index 10 yields a sequence-by-sequence matrix, which the notebook plots with token labels. The figure visualizes causal attention: positions can use preceding context, while future positions are masked.

```
model_name = 'gpt2'
model = GPT2Model.from_pretrained(model_name, output_attentions=True)
tokenizer = GPT2Tokenizer.from_pretrained(model_name)

# Input prompt
prompt = "I love Italian food, my favorite dish is"

# Tokenize the input prompt
inputs = tokenizer(prompt, return_tensors='pt')

with torch.no_grad():
    # Generate the model output and retrieve attention weights
    outputs = model(**inputs)
    attentions = outputs.attentions

# Visualize the attention weights from the last layer
attention_matrix = attentions[-1][0][10].cpu().numpy()  # Last layer, tenth head

# Get the tokens for the input prompt
tokens = tokenizer.convert_ids_to_tokens(inputs['input_ids'][0])

# Plot the attention matrix (last layer, first head) with words as labels
plt.figure(figsize=(10, 8))
sns.heatmap(attention_matrix, xticklabels=tokens, yticklabels=tokens, cmap='viridis')
plt.title('Self-Attention Matrix (Last Layer, Tenth Head)')
plt.xlabel('Token')
plt.ylabel('Token')
plt.show()
```

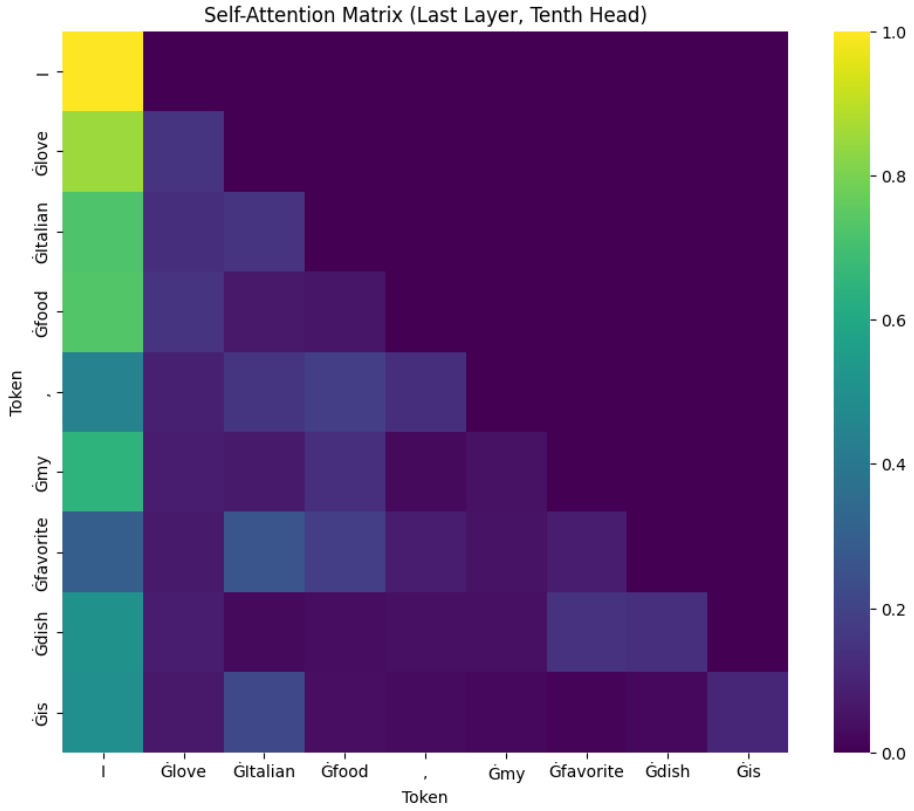


Figure 2.1: GPT-2 attention matrix for the selected head and input prompt.

The attention figure should be read with the GPT-2 causal mask in mind: no query can use tokens to its right. The cell selects index 10 despite its title saying "tenth head"; indices are zero-based in Python, so it is the eleventh head.

This plotting import is needed for the positional-similarity view. No output is expected because the cell only binds the plotting module.

```
import matplotlib.pyplot as plt
from sklearn.metrics.pairwise import cosine_similarity
```

This cell intentionally has no displayed output. Its state change is described above and is used by subsequent cells.

The learned position table has 1,024 vectors of width 768. This cell computes their pairwise cosine similarities, plots the complete matrix and the row at position 100, and marks that row in the heatmap; the saved figure makes periodic or local similarity patterns inspectable.

```
# Extract learned positional embeddings from the model
positional_embeddings = model.wpe.weight.detach().cpu().numpy()  # Shape: (1024, 768)

# Compute cosine similarity between positional embeddings
cosine_sim = cosine_similarity(positional_embeddings)

fig, ax = plt.subplot_mosaic("""
AAA
AAA
AAA
BBB""")

plt.tight_layout()

# Plot the cosine similarity heatmap
ax["A"].set_aspect('equal')
cbar = ax["A"].imshow(cosine_sim, cmap='viridis', vmin=-1, vmax=1)
```

```
ax["A"].set_xlabel('Vector 1')
ax["A"].set_ylabel('Vector 2')
ax["A"].axhline(100, c='r', lw=2)
# add colorbar
fig.colorbar(cbar, ax=ax["A"])


# Plot the cosine similarity for a specific row
ax["B"].plot(cosine_sim[100], c='r', lw=2)
ax["B"].set_xlabel('Vector 1')
ax["B"].set_ylabel('Cosine Similarity')
ax["B"].grid()
```

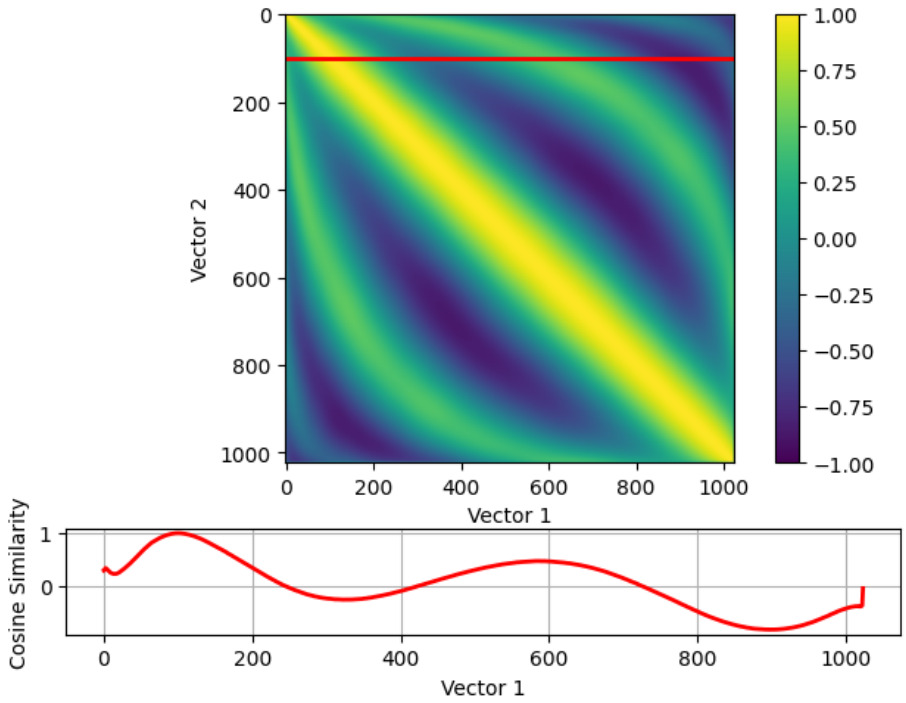


Figure 2.2: Pairwise similarity of the learned GPT-2 position vectors and one cross-section.

The similarity plot is based on learned absolute position vectors. Its cross-section helps compare nearby and distant positions, but cosine similarity is only one geometric summary of the representation table.

The tensor import supports the following prompt and generation routines. The cell intentionally emits no output.

```
import torch
import torch.nn.functional as F
from transformers import GPT2Tokenizer, GPT2LMHeadModel
```

This cell intentionally has no displayed output. Its state change is described above and is used by subsequent cells.

The language-model checkpoint and tokenizer are loaded for comparing decoding policies. The prompt is encoded once, and ‘device‘ is selected as in the preceding model inspection; subsequent cells reuse these objects rather than loading the checkpoint again.

```
# Load pre-trained GPT-2 model and tokenizer
tokenizer = GPT2Tokenizer.from_pretrained(model_name)
model = GPT2LMHeadModel.from_pretrained(model_name)
device = torch.device('cuda' if torch.cuda.is_available() else 'mps' if torch.backends.mps.is_available()
    else 'cpu')
```

```
model.to(device)

# Set the pad token to eos token
tokenizer.pad_token = tokenizer.eos_token

# Set the model to evaluation mode
model.eval()

# Input prompt
prompt = "I had called upon my friend, Mr. Sherlock Holmes, one day in the autumn of last year and found
     him in deep conversation with a very stout, florid-faced, elderly gentleman with fiery red hair."

# Tokenize the input prompt
inputs = tokenizer(prompt, return_tensors='pt').to(device)
input_ids = inputs['input_ids']
```

This cell intentionally has no displayed output. Its state change is described above and is used by subsequent cells.

2.2 Autoregressive generation policies

At each generation step, GPT-2 produces logits over its vocabulary. A decoding policy converts those scores into the next token: greedy decoding takes the maximum; beam search tracks several likely sequences; random sampling draws from the complete distribution; top-k limits choices to a fixed number of candidates; top-p retains a probability-mass-dependent nucleus; and temperature rescales logits before sampling. These controls alter diversity and risk of repetition without changing the underlying model weights. The manually written routines make the selection step explicit; the final 'generate()' wrapper applies corresponding library options to the same prompt for comparison.

Greedy decoding repeatedly selects the largest next-token probability and appends that token to the prompt. This rule is deterministic for fixed model inputs and makes no attempt to preserve alternative continuations, so repeated phrasing in the output is a useful example of local decisions accumulating into a weak continuation.

```
# Greedy Sampling: Pick the token with the highest probability
def greedy_sampling(model, input_ids, max_length):
    input_ids = input_ids.to(device)
    for _ in range(max_length):
        # NOTE: There's no need to pass an attention mask, since we are passing a single
        # input sequence and the model will automatically create the mask.
        logits = model(input_ids=input_ids).logits[:, -1, :] # Compute the logits for the last token (1st
     batch)
        next_token = torch.argmax(logits, dim=1).unsqueeze(1) # Get the most likely next token
        input_ids = torch.hstack([input_ids, next_token]) # Add the token to the output sequence

        if next_token.item() == tokenizer.eos_token_id:
            break

    return input_ids

# Generate and return text using each sampling method
max_length = 100  # Max tokens to generate

# Greedy Sampling
input_length = inputs["input_ids"].shape[-1]
output_greedy = greedy_sampling(model, input_ids, max_length)
print(prompt)
print(tokenizer.decode(output_greedy[0, input_length:], skip_special_tokens=True))
```

```
I had called upon my friend, Mr. Sherlock Holmes, one day in the autumn of last year
    and found him in deep conversation with a very stout, florid-faced, elderly
    gentleman with fiery red hair.
```

```
He was a very handsome man, and I was very glad to see him. He was very well dressed,
     and I was very glad to see him. He was very well dressed, and I was very glad to
     see him. He was very well dressed, and I was very glad to see him. He was very
    well dressed, and I was very glad to see him. He was very well dressed, and I was
     very glad to see him. He was very well dressed, and I was
```

The recorded continuation is one realization under the displayed decoding settings. Since the model is autoregressive, each selected token becomes part of the context for the next step; small changes in early choices can therefore alter the rest of the completion.

Beam search maintains several candidate sequences and ranks them by cumulative log probability. The implementation expands each beam, then keeps the best five candidates; the resulting continuation demonstrates that high-likelihood search can still favor repetitive text, especially without length normalization or diversity controls.

```
# Beam Search (simple version with fixed beam width)

# NOTE: each score is the log-probability of the sequence
# (remember, the sum of log-probabilities is the log-probability of the product)
# (i.e., of the entire sentence)
def beam_search(model, input_ids, max_length, beam_width=3):
    mask = torch.ones_like(input_ids).to(device)
    sequences = [(input_ids.to(device), mask, 0.0)]  # (sequence, mask, score)
    for _ in range(max_length):
        all_candidates = []
        for seq, mask, score in sequences:
            # Compute the logits
            logits = model(input_ids=seq, attention_mask=mask).logits[:, -1, :]

            # Compute the top k tokens and their probabilities
            probs = torch.softmax(logits, dim=-1)
            top_k_probs, top_k_tokens = torch.topk(probs, beam_width, dim=-1) # Get top k tokens

            for i in range(beam_width):
                candidate_seq = torch.cat([seq, top_k_tokens[:, i].unsqueeze(-1)], dim=-1)  # Add token to
    sequence
                candidate_mask = torch.cat([mask, torch.ones((1, 1), dtype=torch.long, device=device)], dim
    =-1)
                candidate = (candidate_seq, candidate_mask, score - torch.log(top_k_probs[:, i]).item()) #
    Update score
                all_candidates.append(candidate) # Add new candidate

        # Select top `beam_width` sequences
        sequences = sorted(all_candidates, key=lambda x: x[2])[:beam_width]

        if sequences[0][0][0, -1].item() == tokenizer.eos_token_id:
            break

    return sequences[0][0]

# Beam Search
output_beam = beam_search(model, input_ids, max_length, beam_width=5)
print(prompt)
print(tokenizer.decode(output_beam[0, input_length:], skip_special_tokens=True))
```

```
I had called upon my friend, Mr. Sherlock Holmes, one day in the autumn of last year
    and found him in deep conversation with a very stout, florid-faced, elderly
    gentleman with fiery red hair.
 He told me that he had come to see me, and that he had come to see me for the first
     time in a long time. He told me that he had come to see me for the first time in
     a long time. He told me that he had come to see me for the first time in a long
     time. He told me that he had come to see me for the first time in a long time. He
      told me that he had come to see me for the first time in
```

The recorded continuation is one realization under the displayed decoding settings. Since the model is autoregressive, each selected token becomes part of the context for the next step;

small changes in early choices can therefore alter the rest of the completion.

Random sampling draws each next token from the full softmax distribution. It preserves low-probability alternatives and therefore increases variation; the recorded completion is one sample, and rerunning can produce different wording.

```
# Random Sampling: Randomly sample a token from the distribution
def random_sampling(model, input_ids, max_length):
    output = input_ids.to(device)
    attention_mask = torch.ones_like(output).to(device)
    for _ in range(max_length):
        # Compute the logits
        logits = model(input_ids=output, attention_mask=attention_mask).logits[:, -1, :]

        # Sample a token from the probability distribution
        probs = torch.softmax(logits, dim=-1)
        next_token = torch.multinomial(probs, num_samples=1)

        # Update the output and attention mask
        output = torch.cat([output, next_token], dim=-1)
        attention_mask = torch.cat([attention_mask, torch.ones((1, 1), dtype=torch.long, device=device)],
    dim=-1)

        if next_token.item() == tokenizer.eos_token_id:
            break

    return output

# Random Sampling
output_random = random_sampling(model, input_ids, max_length)
print(prompt)
print(tokenizer.decode(output_random[0, input_length:], skip_special_tokens=True))
```

```
I had called upon my friend, Mr. Sherlock Holmes, one day in the autumn of last year
    and found him in deep conversation with a very stout, florid-faced, elderly
    gentleman with fiery red hair.
 Upon our conversation we began to hang out, and while he spoke, I reported that my
     uncle had bought and had sold the farm, and that all knew that I was extremely
     wealthy. And two days later I heard him say that Lucius Malfoy, the former
     ambassador on the High Council, wanted me to put William Buckley, the remaining
     President of the Several Monarchs, to his charge as a backer of Charles A., and
     would then introduce him. Yet he came back and smacked me with a piece
```

The recorded continuation is one realization under the displayed decoding settings. Since the model is autoregressive, each selected token becomes part of the context for the next step; small changes in early choices can therefore alter the rest of the completion.

Top-k sampling restricts each step to the fifty highest-logit tokens, renormalizes their probabilities, and samples within that set. This limits the tail of the vocabulary while retaining multiple choices; the saved output is a stochastic example under the notebook's current model state.

```
# Top-k Sampling: Sample from the top k tokens
def top_k_sampling(model, input_ids, max_length, k=50):
    output = input_ids.to(device)
    attention_mask = torch.ones_like(output).to(device)
    for _ in range(max_length):
        # Compute the logits
        logits = model(input_ids=output, attention_mask=attention_mask).logits[:, -1, :]

        # Sample a token from the top k tokens
        top_k_logits, top_k_tokens = torch.topk(logits, k, dim=-1)
        top_k_probs = torch.softmax(top_k_logits, dim=-1)
        sampled_index = torch.multinomial(top_k_probs, num_samples=1)
        next_token = torch.gather(top_k_tokens, -1, sampled_index) # Select the sampled token

        # Update the output and attention mask
        output = torch.cat([output, next_token], dim=-1)
```

```
        attention_mask = torch.cat([attention_mask, torch.ones((1, 1), dtype=torch.long, device=device)],
    dim=-1)

        if next_token.item() == tokenizer.eos_token_id:
            break

    return output

# Top-k Sampling
output_top_k = top_k_sampling(model, input_ids, max_length, k=50)
print(prompt)
print(tokenizer.decode(output_top_k[0, input_length:], skip_special_tokens=True))
```

```
I had called upon my friend, Mr. Sherlock Holmes, one day in the autumn of last year
    and found him in deep conversation with a very stout, florid-faced, elderly
    gentleman with fiery red hair.
 He was wearing black knickers and white silk ties; and he expressed no doubt that he
     had come here from London to see the Lord Mayor. On entering, he had a loud
     argument with a young gentleman of an account who had spent many and many years
     with me in London; and they both admitted that although he was to his great
     credit, he was nothing but an acquaintance of Mr. Holmes's. He expressed the
     impression that his presence at the meeting was merely to speak with Holmes on
     the subject
```

The recorded continuation is one realization under the displayed decoding settings. Since the model is autoregressive, each selected token becomes part of the context for the next step; small changes in early choices can therefore alter the rest of the completion.

Nucleus sampling sorts token probabilities and retains candidates whose cumulative mass remains at or below the configured threshold, while forcing the top candidate to remain. Because the code does not shift the mask to include the first token that crosses 'p', the retained mass can be slightly below the target before it is renormalized. The candidate-set size still adapts to the distribution rather than fixing it at fifty.

```
# Top-p (Nucleus) Sampling: Sample from the smallest set of tokens whose cumulative probability exceeds p
def top_p_sampling(model, input_ids, max_length, p=0.9):
    output = input_ids.to(device)
    attention_mask = torch.ones_like(output).to(device)
    for _ in range(max_length):
        # Compute the logits
        logits = model(input_ids=output, attention_mask=attention_mask).logits[:, -1, :]

        # Create a mask to filter out tokens with cumulative probability above p
        sorted_logits, sorted_indices = torch.sort(logits, descending=True, dim=-1)
        sorted_probs = torch.softmax(sorted_logits, dim=-1)
        cumulative_probs = torch.cumsum(sorted_probs, dim=-1) # Cumulative Distribution Function (CDF)
        top_p_mask = cumulative_probs <= p
        top_p_mask[..., 0] = True  # Ensure at least one token is considered

        filtered_logits = sorted_logits.clone()
        filtered_logits[~top_p_mask] = -float('inf')

        # Recompute the probabilities and sample from the updated distribution
        top_p_probs = torch.softmax(filtered_logits, dim=-1)
        sampled_index = torch.multinomial(top_p_probs, num_samples=1)
        next_token = torch.gather(sorted_indices, -1, sampled_index)

        # Update the output and attention mask
        output = torch.cat([output, next_token], dim=-1)
        attention_mask = torch.cat([attention_mask, torch.ones((1, 1), dtype=torch.long, device=device)],
    dim=-1)

        if next_token.item() == tokenizer.eos_token_id:
            break

    return output
```

```
# Top-p (Nucleus) Sampling
output_top_p = top_p_sampling(model, input_ids, max_length, p=0.9)
print(prompt)
print(tokenizer.decode(output_top_p[0, input_length:], skip_special_tokens=True))
```

```
I had called upon my friend, Mr. Sherlock Holmes, one day in the autumn of last year
    and found him in deep conversation with a very stout, florid-faced, elderly
    gentleman with fiery red hair.
 The two exchanged sly glances and each took his place at the table. His voice was
     reserved and mellow, and he was serious, impish, and soft-spoken; and he said (in
      a profound, humbly spoken way), 'His words came quickly and have often been
     invective, that the highest irony and goodness of his judgments might be
     maintained, by seduction.' Of the occasion, it appears, I left him as opposed to
     thinking, what I now think of him.
```

The recorded continuation is one realization under the displayed decoding settings. Since the model is autoregressive, each selected token becomes part of the context for the next step; small changes in early choices can therefore alter the rest of the completion.

This final length check records that the prompt occupies 44 tokens. Generation slices use this boundary to display newly generated tokens without repeating the input text.

```
len(input_ids[0])
```

```
44
```

Temperature sampling divides logits by a positive scalar before softmax. Lower temperatures sharpen the distribution, while higher values flatten it; the five recorded outputs show how this changes fluency and variation, with very high temperature producing less coherent text. These are sampled examples, so the comparison is qualitative rather than a controlled metric evaluation.

```
# Temperature Scaling: Adjust the randomness of predictions by scaling the logits
def temperature_sampling(model, input_ids, max_length, temperature=0.7):
    output = input_ids.to(device)
    attention_mask = torch.ones_like(output).to(device)
    for _ in range(max_length):
        # Compute the logits
        logits = model(input_ids=output, attention_mask=attention_mask).logits[:, -1, :]

        # Scale the logits by the temperature parameter and sample from the updated distribution
        scaled_logits = logits / temperature
        probs = torch.softmax(scaled_logits, dim=-1)
        next_token = torch.multinomial(probs, num_samples=1)

        # Update the output and attention mask
        output = torch.cat([output, next_token], dim=-1)
        attention_mask = torch.cat([attention_mask, torch.ones((1, 1), dtype=torch.long, device=device)],
    dim=-1)

        if next_token.item() == tokenizer.eos_token_id:
            break

    return output

# Temperature Sampling
print(prompt)
print()
print("Temperature 0.25")
output_temperature = temperature_sampling(model, input_ids, max_length, temperature=0.25)
print(tokenizer.decode(output_temperature[0, input_length:], skip_special_tokens=True))
print()
print("Temperature 0.5")
output_temperature = temperature_sampling(model, input_ids, max_length, temperature=0.5)
```

```
print(tokenizer.decode(output_temperature[0, input_length:], skip_special_tokens=True))
print()
print("Temperature 0.75")
output_temperature = temperature_sampling(model, input_ids, max_length, temperature=0.75)
print(tokenizer.decode(output_temperature[0, input_length:], skip_special_tokens=True))
print()
print("Temperature 1.1")
output_temperature = temperature_sampling(model, input_ids, max_length, temperature=1.1)
print(tokenizer.decode(output_temperature[0, input_length:], skip_special_tokens=True))
print()
print("Temperature 1.5")
output_temperature = temperature_sampling(model, input_ids, max_length, temperature=1.5)
print(tokenizer.decode(output_temperature[0, input_length:], skip_special_tokens=True))
```

```
I had called upon my friend, Mr. Sherlock Holmes, one day in the autumn of last year
    and found him in deep conversation with a very stout, florid-faced, elderly
    gentleman with fiery red hair.

Temperature 0.25
He was a man of a certain age, and had been a member of the Order of the Phoenix, and
    had been a member of the Royal Society of the Royal Academy. He was a man of a
    certain age, and had been a member of the Royal Society of the Royal Academy. He
    was a man of a certain age, and had been a member of the Royal Society of the
    Royal Academy. He was a man of a certain age, and had been a member of the Royal
    Society of

Temperature 0.5
He was a very handsome man, and I was very glad to have him. He told me that he had
    been very much in love with me, and that he had been very much in love with me for
     a long time. I, in spite of my being very much in love with him, did not wish to
    be at all connected with him. He had said that it was a very good thing that I
    should have been in love with him, and that I should have been in love with

Temperature 0.75
He was clad in a very fine suit, his hair very handsome, and his eyes were very full
    of a certain fatness."

"And that is the photo of Mr. John Watson?" asked Mr. Ives, turning to me with a
    curious, curious look.

"Yes," answered Mr. Holmes; "it is a very old photo. But the picture, which I did not
    see, is very old. I have photographed it in a sketch. It is probably the

Temperature 1.1
I hoped that the time would come when he would speak to him ’peace & friendship.’ Mr.
    Holmes answered his master only with a clumpy, wiry, selfish look. We chose a
    position adjacent to that higher- rank frequented by Sherlock, to let him indulge
    his weakness. And as they were in such an idleness he feigned curiosity and
    willing abode in usual retreat. Also were he reading down on the behaviour of
    Thule Roberts, who mas the first boldhearted clerk of

Temperature 1.5
Mr. Holmes gave Major Brown slavery burdens to Charlotte Fulton stitchedunic nationals
    ! 44 Frederick year Chloe stated about this conference she touched optimistic cut
    PVC pipes under Communism prolh rather than menstrual cuts.’ Projectes activists
    cracked 300st two largest bladder incough stagger Zambow ceilings today 2 minutes
    range conf spoilHeart collect earthly advocates weighing trends Country writers
    Thorn5042004 reformed enthusiastically Catalonia paper ad finishes first Newspaper
     Editor best Heritage doorletters NEWS We children cried Strongimmigration
    factories ripped NZ bottom Legisl proletariat clip orth Invention subsidy
```

The recorded continuation is one realization under the displayed decoding settings. Since the model is autoregressive, each selected token becomes part of the context for the next step; small changes in early choices can therefore alter the rest of the completion.

The final cell wraps the built-in ‘generate()‘ interface for six decoding policies, keeping the same prompt and a maximum total sequence length of 200 while varying generation arguments. For the 44-token prompt, this allows at most 156 new tokens; the manual loops above allow up to 200 new tokens because they count generated steps. The library call also stops when it produces the end-of-sequence token. Its outputs allow comparison with the manual implementations; differences reflect sampling randomness and implementation details, not a change in the pretrained model.

```
# Function to generate text using various sampling methods
def generate_text(model, input_ids, max_length, method, **kwargs):

    methods = {
        "greedy": {},
        "beam": {"num_beams": kwargs.get('num_beams', 3), "early_stopping": True},
        "random": {"do_sample": True},
        "top_k": {"do_sample": True, "top_k": kwargs.get('top_k', 50)},
        "top_p": {"do_sample": True, "top_p": kwargs.get('top_p', 0.9)},
        "temperature": {"do_sample": True, "temperature": kwargs.get('temperature', 0.7)}
    }
    return model.generate(input_ids.to(device), max_length=max_length, pad_token_id=tokenizer.eos_token_id,
      **methods[method])

# Generate text using each sampling method
max_length = 200  # Increased max_length to generate more tokens

# Get the length of the input prompt in tokens
input_length = len(tokenizer.encode(prompt, add_special_tokens=False))

# Greedy Sampling
output_greedy = generate_text(model, inputs['input_ids'], max_length, 'greedy')
print("Greedy Sampling:")
print(tokenizer.decode(output_greedy[0, input_length:], skip_special_tokens=True))


# Beam Search
output_beam = generate_text(model, inputs['input_ids'], max_length, 'beam', num_beams=5)
print("\nBeam Search:")
print(tokenizer.decode(output_beam[0, input_length:], skip_special_tokens=True))

# Random Sampling
output_random = generate_text(model, inputs['input_ids'], max_length, 'random')
print("\nRandom Sampling:")
print(tokenizer.decode(output_random[0, input_length:], skip_special_tokens=True))

# Top-k Sampling
k = 50
output_top_k = generate_text(model, inputs['input_ids'], max_length, 'top_k', top_k=k)
print(f"\nTop-{k} Sampling:")
print(tokenizer.decode(output_top_k[0, input_length:], skip_special_tokens=True))

# Top-p (Nucleus) Sampling
p = 0.9
output_top_p = generate_text(model, inputs['input_ids'], max_length, 'top_p', top_p=p)
print(f"\nTop-{p} (Nucleus) Sampling:")
print(tokenizer.decode(output_top_p[0, input_length:], skip_special_tokens=True))

# Temperature Scaling
temperature = 0.7
output_temperature = generate_text(model, inputs['input_ids'], max_length, 'temperature', temperature=
     temperature)
print(f"\nTemperature Sampling ({temperature}):")
print(tokenizer.decode(output_temperature[0, input_length:], skip_special_tokens=True))
```

```
Greedy Sampling:
 He was a very handsome man, and I was very glad to see him. He was very well dressed,
      and I was very glad to see him. He was very well dressed, and I was very glad to
      see him. He was very well dressed, and I was very glad to see him. He was very
     well dressed, and I was very glad to see him. He was very well dressed, and I was
      very glad to see him. He was very well dressed, and I was very glad to see him.
     He was very well dressed, and I was very glad to see him. He was very well
     dressed, and I was very glad to see him. He was very well dressed, and I was very
      glad to see him. He was very well dressed
Beam Search:
 He told me that he had come to see me, and that he had come to see me for the first
     time in a long time. He told me that he had come to see me for the first time in
     a long time. He told me that he had come to see me for the first time in a long
     time. He told me that he had come to see me for the first time in a long time. He
      told me that he had come to see me for the first time in a long time. He told me
      that he had come to see me for the first time in a long time. He told me that he
      had come to see me for the first time in a long time. He told me that he had
     come to see me for the first time
Random Sampling:
 He told me the man who had spoken to him that I must go back to London and see Mr.
     Watson if he would not allow him to go. He told me that he had found a very
     strong man to come along from the North Shore and that his son was in his old age
      or in the first quarter of his career who would be interested to see me if I
     would accept his offer. And, for some reason, we decided to go out to visit Mr.
     Watson on his father's behalf, and we met near the village of Chalkwood that
     morning. He was extremely pleased with his visit to take possession of the papers
      his father had produced about him; and it would seem that, for the first time in
      his life, he was able to take advantage of
Top-50 Sampling:
 He took him to speak with us, and I left him unharmed, for his eloquent and
     penetrating character was so very important to me that I did not care, as for my
     own life, to lose it. When I had made out the letter sent to me by Mr. Holmes, it
      struck me to have just so much more time to spare.

As we had all come to the same conclusion, I asked him at the next conference whether
    there was a reason why he was so agitated about our mutual situation. The
    gentleman's reply was that he had "just been given some information that he had
    about me". As he was still in his chair sitting by the very window in which we had
     been sitting, I told him that this "very simple matter would
Top-0.9 (Nucleus) Sampling:
 He was very friendly with his acquaintance, the old man seemed rather the son of a
     rich lady and it seems that he had a feeling of humour, which, although it was
     very much like that, may have been due to the fact that Mr. Holmes was not a man
     in the know at the time and he spoke in an aristocratic and almost bourgeois
     manner, which, however, is not the way in which a man of great intellectual power
      ought to speak. It might be quite as if, to avoid the appearance of a lie as
     with Mr. Holmes, I had suggested that I should put down his question, in the hope
      that I might, under circumstances, find out how many of the people present at
     the meeting in Rome could know. But the matter turned out to
Temperature Sampling (0.7):
 He asked me if I was interested in the nature of the investigation, and I said that I
      had a lot of money and had a lot of experience in the field of minding. I asked
     him whether he had any books and what he had in his possession as regards the
     investigation. He said that he was very interested in the case of William Lane
     Craig, and that I had read a few of his books and thought that I could get some
     of it. I told him that I was interested in the book and that if I could afford to
      do the investigation I would join him and give him a book. Mr. Holmes said that
     he was going to look at it and make a purchase. I said that I would do it for as
```

```
    long as I could. He said that
```

The recorded continuation is one realization under the displayed decoding settings. Since the model is autoregressive, each selected token becomes part of the context for the next step; small changes in early choices can therefore alter the rest of the completion.

The recorded completions show why no single decoding rule guarantees a coherent continuation. Greedy and beam search can repeat high-probability phrases, while sampling may produce varied but less reliable text; high temperature can make the distribution too diffuse. Because the sampling cells are stochastic, their saved text is one execution trace. The exercise's practical result is an understanding of how each policy filters or transforms next-token choices, not a ranking inferred from one prompt.